

AUTONOMOUS SURVEILLANCE VEHICLE

BY

SANKEETH KURIAN

ABSTRACT

The project is designed so that a mobile robot represents a very attractive and multidisciplinary field with a lot of applications and scientific research possibilities. Today, security is a major concern for many businesses. Autonomous security robots are automated security & surveillance robots that combine and develop multitude of different technologies to perform security & surveillance operations such as reporting, monitoring, investigating, and intruder detection. Autonomous security robots have the ability to move in complex environments, detecting and reporting anomalies in the environment. Their sensors and cameras allow them not to lose their balance, even when moving through complex terrain. A remote-controlled surveillance robot is any robot that is remotely controlled to capture images/video for specific purposes. Mobile robots that are controlled remotely have important roles in area of rescue and military. Security robots automatically move along the patrol route, detect strangers, and transmit messages to the guard. The operator behind the monitor screen is already in the past, the robots themselves detect the intrusion. A security guard will only be required to physically counteract a potential intruder. The development of microcomputers, single board computers and embedded systems has helped to deploy low-cost solutions for this domain. In this project it's proposed such a low-cost mobile robot platform with fixed four-wheel chassis. The mobile robot has the ability to move as directed by the controller. The goal of this project is to help you begin programming with Python to control our robots with Jetson Nano board. We will go over the basics and some simple examples to control your robot from a Python GUI Jetson Nano board.

Contents

1	INTRODUCTION	5
2	LITERATURE SURVEY	8
2.1	Donato Di Paola and Annalisa Milella	8
2.2	Sebastián Valladares and Mayerly Toscano	8
2.3	Stefan Witwicki and Jose Carlos Castillo	8
3	CHALLENGES IN MOBILE ROBOTICS	9
4	SYSTEM MODEL	11
5	IMPLEMENTATION	12
5.1	Regression	12
5.2	Classification	12
5.3	Tensor RT optimization	12
6	WORKING	14
6.1	Road Following	14
6.1.1	Data Collection	14
6.1.2	Training	14
6.2	Collision Avoidance	14
6.2.1	Data Collection	14
6.2.2	Training	15
6.3	Road Following and Collision Avoidance	15
6.4	Object Detection	15
7	HARDWARE DESCRIPTION	16
7.1	PCA9685 16-CHANNEL MODULE	16
7.2	L298N DRIVER	16
7.3	DC Motor	16
8	SOFTWARE DESCRIPTION	17
8.1	Software Requirements	17
8.1.1	Jupyter Notebook	17
8.1.2	Python	17
9	EXPERIMENTAL ANALYSIS AND RESULTS	19
9.1	CODE	19
9.2	RESULT	25
9.3	PERFORMANCE EVALUATION	25

List of Figures

1.1	Nvidia Jetson Nano (a). Board Description (b). Interfaced camera module	6
1.2	Nvidia Jetson Nano Pin diagram	7
3.1	Proposed body of the Mobile Robot	9
3.2	Major challenges of Mobile Robot	10
4.1	System Model	11
5.1	Tensor RT optimization	13
9.1	Results	25

Chapter 1

INTRODUCTION

An Autonomous robot vehicle is capable of moving from one place to another and recognize its surroundings and basic tools & objects. It is a field of the computer science that deals with the analysis and understanding of images and videos focused on mobile devices (e.g., smartphones, tablets) which is quite similar to surveillance cameras, but provides automatic image capturing, evaluation and processing capabilities. It primarily enables a computer to recognize and evaluate images. It is similar to voice recognition technology, but uses images instead. The mobile robot in our Mini-project is a remote controlled one which recognize objects, tools and people in the surrounding from the camera attached in to it.

1. The camera acts as a feedback element that is capable of recognizing its surrounding.
2. The camera in the robot uses the AI technology. It uses AI programs to wisely deal with images and videos. The platform used for AI programming is TensorFlow, an end-to-end open source.
3. TensorFlow provides a collection of workflows to develop and train models using Python or JavaScript. Here we use Python as the programming language to simulate the robot.

NVIDIA Jetson Nano Developer Kit is a small, powerful computer that lets you run multiple neural networks in parallel for applications like image classification, object detection, segmentation, and speech processing. All in an easy-to-use platform that runs in as little as 5 watts.

Why Jetson Nano instead of Raspberry Pi?

In terms of GPU, the Jetson Nano is a little ahead because of their 128- core Maxwell GPU @ 921 MHz. The Raspberry Pi 4 GPU is weaker compared to the Jetson Nano. In the case of CPU, the Raspberry uses the latest and best CPU, the Quad-core ARM cortex-A72 64-bit @ 1.5 GHz. The key features of Jetson Nano include:

- GPU: 128-core NVIDIA Maxwell™ architecture-based GPU
- CPU: Quad-core ARM® A57
- Video: 4K @ 30 fps (H.264/H.265) / 4K @ 60 fps (H.264/H.265) encode and decode
- Camera: MIPI CSI-2 DPHY lanes, 12x (Module) and 1x (Developer Kit)

- Memory: 4 GB 64-bit LPDDR4; 25.6 gigabytes/second
- Connectivity: Gigabit Ethernet
- OS Support: Linux for Tegra®
- Module Size: 70mm x 45mm
- Developer Kit Size: 100mm x 80mm

Raspberry pi V2 camera module is interfaced with the Board with a resolution - 8 megapixel native resolution sensor. A 15 cm ribbon cable attached to the module slots straight into your MIPI Camera Serial Interface port (MIPI CSI). The Raspberry Pi Camera Board is a designed add-on module for its corresponding hardware. It attaches to the hardware through a custom CSI interface. The sensor has 8 megapixel native resolution in still capture mode. In video mode it supports capture resolutions up to 1080p at 30 frames per second. The Camera specifications include:

- Fixed focus lens on-board
- Improved resolution - 8 megapixel native resolution sensor-capable of 3280 x 2464 pixel static images
- Supports 1080p30, 720p60 and 640x480p90 video
- Size 25mm x 23mm x 9mm
- Weight just over 3g

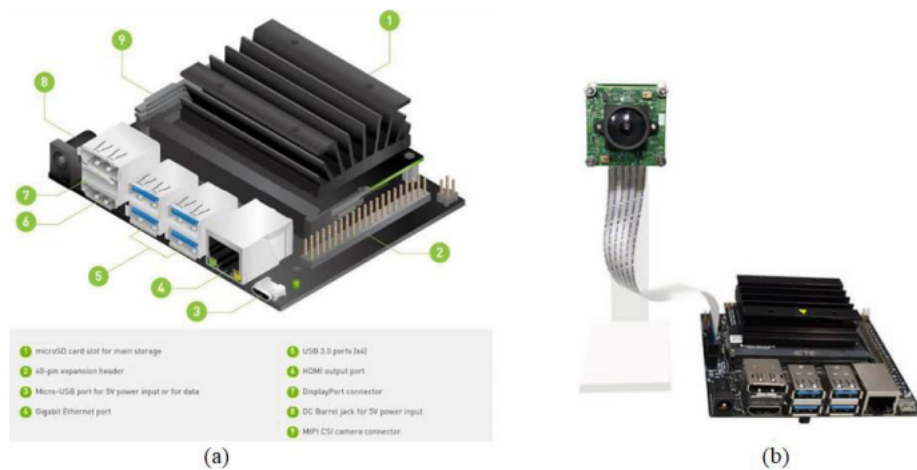


Figure 1.1: Nvidia Jetson Nano (a). Board Description (b). Interfaced camera module

Jetson Nano J41 Header					
Sysfs GPIO	Name	Pin	Pin	Name	Sysfs GPIO
	3.3 VDC Power	1	2	5.0 VDC Power	
	I2C_2_SDA I2C Bus 1	3	4	5.0 VDC Power	
	I2C_2_SCL I2C Bus 1	5	6	GND	
gpio216	AUDIO_MCLK	7	8	UART_2_TX /dev/ttyTHS1	
	GND	9	10	UART_2_RX /dev/ttyTHS1	
gpio50	UART_2_RTS	11	12	I2S_4_SCLK	gpio79
gpio14	SPI_2_SCK	13	14	GND	
gpio194	LCD_TE	15	16	SPI_2_CS1	gpio232
	3.3 VDC Power	17	18	SPI_2_CS0	gpio15
gpio16	SPI_1_MOSI	19	20	GND	
gpio17	SPI_1_MISO	21	22	SPI_2_MISO	gpio13
gpio18	SPI_1_SCK	23	24	SPI_1_CS0	gpio19
	GND	25	26	SPI_1_CS1	gpio20
	I2C_1_SDA I2C Bus 0	27	28	I2C_1_SCL I2C Bus 0	
gpio149	CAM_AF_EN	29	30	GND	
gpio200	GPIO_PZ0	31	32	LCD_BL_PWM	gpio168
gpio38	GPIO_PE6	33	34	GND	
gpio76	I2S_4_LRCK	35	36	UART_2_CTS	gpio51
gpio12	SPI_2_MOSI	37	38	I2S_4_SDIN	gpio77
	GND	39	40	I2S_4_SDOUT	gpio78

Figure 1.2: Nvidia Jetson Nano Pin diagram

Chapter 2

LITERATURE SURVEY

2.1 Donato Di Paola and Annalisa Milella

Donato has presented his publication on Autonomous Mobile Robotic System for Surveillance of Indoor Environments in Hitachi Ltd. Tokyo, Japan in 2010. Annalisa Milella, a researcher at Italian National Research Council has contributed her partial work in the field of Surveillance Indoor Environments.

2.2 Sebastián Valladares and Mayerly Toscano

Sebastián Valladares has presented a performance Evaluation of the Nvidia Jetson Nano through a Real-Time Machine Learning Application in January 2021 along with Mayerly Toscano's research while affiliated with Universidad Politécnica.

2.3 Stefan Witwicki and Jose Carlos Castillo

Stefan Witwicki and Jose Carlos Castillo (Sept 2017) has presented their work on Decision-Making Framework for Networked Multiagent Systems in IEEE Robotics & Automation Magazine.

Chapter 3

CHALLENGES IN MOBILE ROBOTICS

The increase of navigation autonomy in mobile robots creates benefits for companies that use them. But such a system has its challenges. Developing these systems with off-the-shelf hardware and sensors is essential to their growth.

The major challenges include mechanical structure, navigation, and human-centered intelligent control, of which navigation is the most challenging functionality required for such autonomous systems. The navigation comprises four dominant blocks of competencies: perception, localization, cognition and motion control.

To function properly, an autonomous mobile robot (AMR) must navigate safely through known and unknown environments while carrying out its assigned task. The robot needs to build a model or map, estimate its current position, and navigate to target points. Mapping, localization, and navigation are classic problems for mobile robots. Finding a good solution to these problems is needed as robots become totally autonomous, adaptable to changing circumstances, and expand their range of application.

Three challenges focus on fundamental problems in robotics: developing robot swarms, improving navigation and exploration, and developing artificial intelligence that can “learn how to learn”, and use common sense to make moral and social decisions.

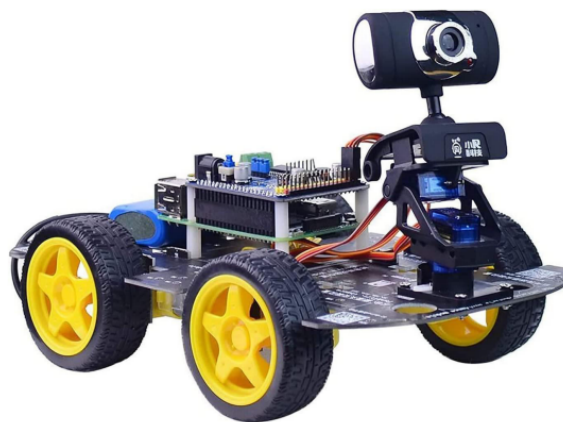


Figure 3.1: Proposed body of the Mobile Robot

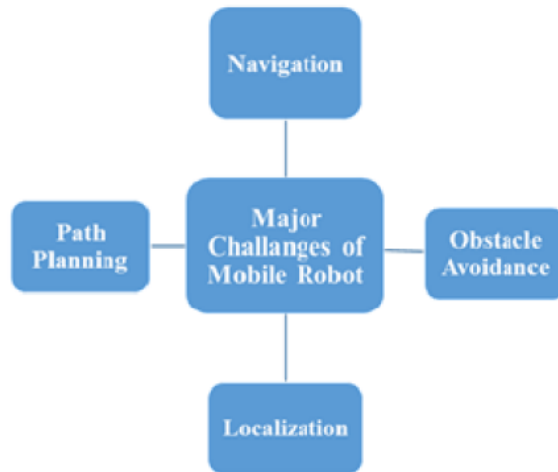


Figure 3.2: Major challenges of Mobile Robot

The diagram illustrates the primary challenges faced by mobile robots, categorized into key areas that are essential for their effective operation. These challenges include:

- **Navigation:** Involves path finding and map creation to guide the robot through its environment.
- **Path Planning:** Focuses on determining the optimal route and avoiding collisions with obstacles.
- **Obstacle Avoidance:** Requires real-time detection and dynamic adjustments to navigate around impediments.
- **Localization:** Entails determining the robot's position and orientation in real-time, often in dynamic environments.

Chapter 4

SYSTEM MODEL

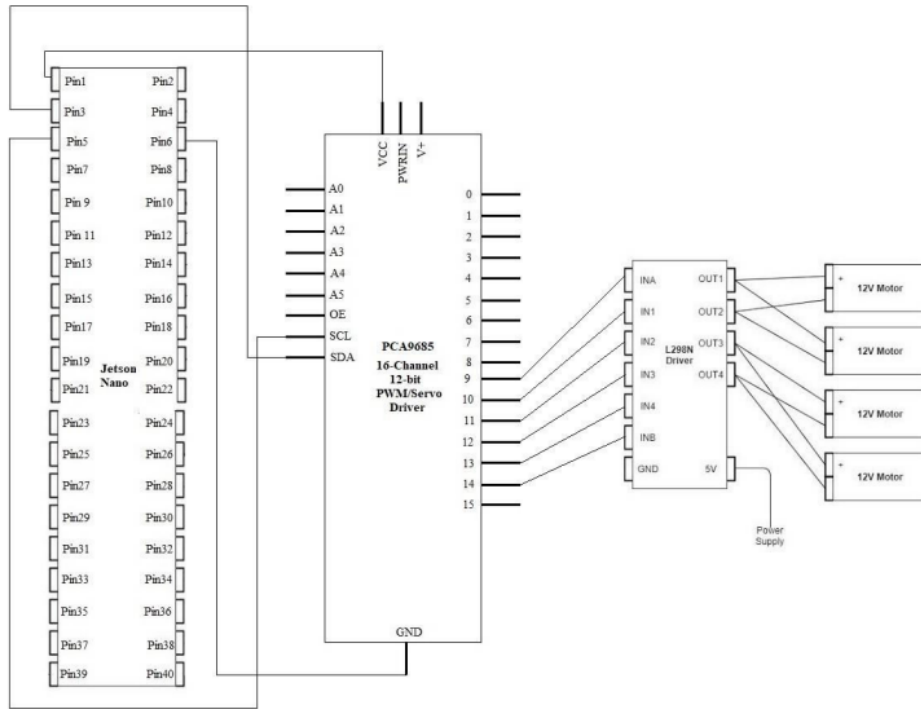


Figure 4.1: System Model

Figure 4.1 shows the proposed system, which uses a small powerful computer (Jetson Nano board) for embedded applications and AI IoT that delivers the power of modern AI. The above systems model describes the process-oriented representation that emphasizes the flow of information between modules. It also describes how processes interact and what operations these processes perform.

System Model Description: As from Figure 4.1, it's clear that the pin1 of Jetson board is connected to VCC of PCA, Pin3 and Pin5 is connected to SDA and SCL respectively. The Pin6 is then grounded to PCA. The input of the driver (L298N) is interfaced with its preceding PCA. The two-output port of the driver is connected to the input of the four DC motors in a parallel way. These DC motors then drives the robot.

Chapter 5

IMPLEMENTATION

5.1 Regression

Regression is a technique for investigating the relationship between independent variables or features and a dependent variable or outcome. It's used as a method for predictive modelling in machine learning, in which an algorithm is used to predict continuous outcomes. Robot is taught to detect a target x, y image coordinate that the Robot will chase. As Robot gets closer to the point, it moves further along the track.

5.2 Classification

The Classification algorithm is a Supervised Learning technique that is used to identify the category of new observations on the basis of training data. In Classification, a program learns from the given dataset or observations and then classifies new observation into a number of classes or groups. Such as, Yes or No, 0 or 1, Spam or Not Spam, cat or dog, etc. Classes can be called as targets/labels or categories. Robot to detect two scenarios free and blocked. We use this AI classifier to prevent Robot from entering dangerous territory. This project focuses on combining both optimized regression and classification models to enable the Robot to follow a specific path on the track and at the same time also be able to avoid collisions that come on its way in real-time.

5.3 Tensor RT optimization

NVIDIA TensorRT is used for efficiently deploying neural networks onto the embedded Jetson platform, improving performance and power efficiency using graph optimizations, kernel fusion, and FP16/INT8 precision. Vision primitives, such as imageNet for image recognition, detectNet for object detection, segNet for semantic segmentation, and poseNet for pose estimation inherit from the shared tensorNet object. NVIDIA TensorRT-based applications perform up to 36X faster than CPU-only platforms during inference, enabling developers to optimize neural network models trained on all major frameworks, calibrate for lower precision with high accuracy, and deploy to hyperscale data centers, embedded platforms, or automotive product platforms.

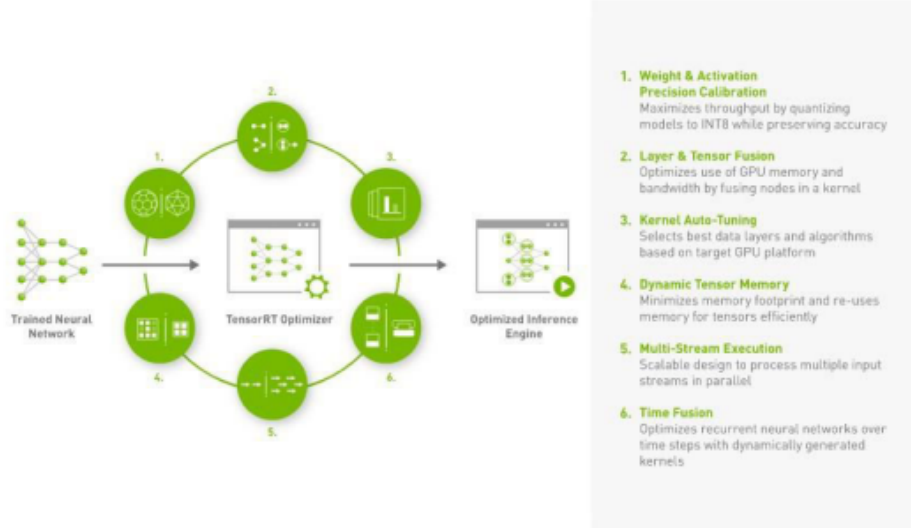


Figure 5.1: Tensor RT optimization

The figure illustrates the comprehensive TensorRT optimization pipeline for enhancing neural network inference on NVIDIA hardware, such as the Jetson Nano used in the mobile robot project. At its core, the diagram depicts a cyclical workflow beginning with a trained neural network model, which undergoes initial calibration for precision—either in INT8 (8-bit integer) or FP16 (half-precision floating-point) formats—to balance accuracy and computational efficiency. This step is crucial for resource-constrained embedded systems like autonomous security robots, where maintaining high inference speeds without significant accuracy loss is essential for real-time tasks such as object detection and anomaly reporting.

Following calibration, the process advances to layer and tensor analysis, where the model is profiled to identify memory usage and computational bottlenecks across its layers. The TensorRT engine then intervenes by optimizing these elements: it fuses compatible layers to reduce overhead, selects efficient kernel implementations from a vast library, and applies dynamic memory optimizations like footprint reduction and reuse. These transformations generate a streamlined, hardware-specific engine that minimizes latency and maximizes throughput, making it particularly suitable for the Jetson Nano’s 128-core Maxwell GPU in handling AI-driven surveillance operations.

Finally, the optimized engine supports deployment with features like multi-stream execution for parallel processing and recurrent neural network (RNN) optimizations over kernels, enabling scalable inference in dynamic environments. This end-to-end approach not only accelerates model performance—often achieving up to 40x speedup over unoptimized frameworks like TensorFlow—but also ensures compatibility with the project’s Python-based control systems, allowing seamless integration for robot navigation, intruder detection, and video analytics on low-cost platforms.

Chapter 6

WORKING

6.1 Road Following

This section outlines the process of enabling the robot to follow a predetermined path using a trained neural network.

6.1.1 Data Collection

- Place the robot in various positions on a path (e.g., offset from center, different angles).
- Display the live camera feed from the robot.
- Using a gamepad controller, place a 'green dot' on the image, corresponding to the target direction for the robot to travel.
- Store the X, Y coordinates of the green dot along with the corresponding image from the robot's camera. Train a neural network to predict the X, Y values of the label, using an approximate steering value computed from the predicted coordinates.

6.1.2 Training

- Train a neural network to process an input image and output a set of X, Y values corresponding to a target, using the PyTorch deep learning framework with the ResNet18 neural network architecture for the road follower application.
- Optimize the trained model using TensorRT to enhance performance.

6.2 Collision Avoidance

This section details the steps to equip the robot with the ability to detect and avoid obstacles.

6.2.1 Data Collection

- Place the robot in scenarios where its "safety bubble" is violated, label these as 'blocked', and save a snapshot of the robot's view with this label.

- Manually place the robot in safe scenarios where it can move forward, label these as 'free', and save a snapshot of the robot's view with this label.

6.2.2 Training

- Train an image classifier to detect the two classes 'free' and 'blocked' for collision avoidance, utilizing the PyTorch deep learning library.
- Optimize the trained model using TensorRT to improve inference speed and efficiency.

6.3 Road Following and Collision Avoidance

This section integrates the previously developed models to enable simultaneous path following and obstacle avoidance.

- Combine the optimized regression and classification models to allow the robot to perform road following while avoiding collisions with obstacles in real-time.
- Execute the Python program using a Jupyter notebook for interactive development and testing.

6.4 Object Detection

This section covers the implementation of object detection capabilities using a pre-trained model.

- Download the pre-trained object detection model, DetectNet.
- Execute the pre-trained model; if any trained objects are within the camera's field of view, record their presence.

Chapter 7

HARDWARE DESCRIPTION

7.1 PCA9685 16-CHANNEL MODULE

PCA9685 16-Channel module is very useful and can help a lot when there are insufficient PWM output pins. The most important application is to control servo motors. This module can control a large number of servo motors at the same time. Also the PCA9685 driver can be used to control 16 servo motors. In order to control DC Motors, we use Pulse Width Modulation (PWM). The Jetson Nano has the capability of generating two PWM pulses on the J41 Header. However, you need to reconfigure the device tree to make these signals available on Pins 32 and 33. A simpler way is to use an external piece of hardware. For our demo, we use a PCA9685 12-bit PWM/ DC Driver. Using only two pins, we can control 16 free-running PWM outputs. You can even chain up 62 breakouts to control up to 992 PWM outputs.

7.2 L298N DRIVER

The L298N is a dual H-Bridge motor driver which allows speed and direction control of two DC motors at the same time. The module can drive DC motors that have voltages between 5 and 35V, with a peak current up to 2A. An H-Bridge circuit contains four switching elements, transistors or MOSFETs, with the motor at the center forming an H-like configuration. By activating two particular switches at the same time we can change the direction of the current flow, thus change the rotation direction of the motor. So if we combine these two methods, the PWM and the H Bridge, we can have a complete control over the DC motor.

7.3 DC Motor

A DC motor is a rotary motor that converts direct current (DC) into mechanical energy or, conversely, converts mechanical energy into DC power. This motor is integral to the robot's mobility and control system. The DC motor used in this project has the following specifications:

- Voltage: 12 V
- Speed: 300 rpm

Chapter 8

SOFTWARE DESCRIPTION

8.1 Software Requirements

To achieve optimal results during the project, the system relies on specific software tools. This section provides detailed information about the software utilized in the development and operation of the robot.

8.1.1 Jupyter Notebook

JupyterLab is a modern, web-based interactive development environment designed for notebooks, code, and data. Its highly flexible interface allows users to customize and organize workflows, making it ideal for applications in data science, scientific computing, computational journalism, and machine learning. Featuring a modular design, JupyterLab supports extensions to enhance its functionality. In this project, it is employed to execute Python codes and enable wireless control of the robot via a server.

8.1.2 Python

Python is an interpreted, high-level, general-purpose programming language, developed by Guido van Rossum and first released in 1991. Its design philosophy emphasizes code readability, prominently utilizing significant whitespace. With its language constructs and object-oriented approach, Python facilitates the creation of clear, logical code for both small and large-scale projects. It is dynamically typed and garbage-collected, supporting multiple programming paradigms, including procedural, object-oriented, and functional programming.

Libraries Used

The following libraries are integral to the Python programs developed for this project:

- **TensorFlow:** A free and open-source software library for dataflow and differentiable programming across various tasks. As a symbolic math library, it is widely used for machine learning applications, including neural networks. It serves both research and production purposes at Google.
- **PyTorch:** An open-source machine learning framework based on the Python programming language and the Torch library. It is a preferred platform for deep learning research, designed to accelerate the transition from research prototyping to deployment.

- **OpenCV (CV2):** An open-source computer vision and machine learning software library. Originally developed to provide a common infrastructure for computer vision applications and to enhance the use of machine perception in commercial products, OpenCV offers over 2500 optimized algorithms. These include a comprehensive set of classic and state-of-the-art computer vision and machine learning algorithms, enabling tasks such as face detection and recognition, object identification, human action classification in videos, camera movement tracking, moving object tracking, 3D model extraction, stereo camera point cloud generation, image stitching for high-resolution scene reconstruction, image database similarity searches, red-eye removal from flash photography, eye movement tracking, scenery recognition, and augmented reality marker placement.

Chapter 9

EXPERIMENTAL ANALYSIS AND RESULTS

9.1 CODE

```
1 # 1. Road Following sliders
2 speed_control_slider = ipywidgets.FloatSlider(min=0.0, max=1.0, step
    =0.01, description='speed control')
3 steering_gain_slider = ipywidgets.FloatSlider(min=0.0, max=1.0, step
    =0.01, value=0.04, description='steering gain')
4 steering_dgain_slider = ipywidgets.FloatSlider(min=0.0, max=0.5, step
    =0.001, value=0.0, description='steering kd')
5 steering_bias_slider = ipywidgets.FloatSlider(min=-0.3, max=0.3, step
    =0.01, value=0.0, description='steering bias')
6
7 display(speed_control_slider, steering_gain_slider,
    steering_dgain_slider, steering_bias_slider)
8
9 # Collision Avoidance sliders
10 blocked_slider = ipywidgets.FloatSlider(min=0.0, max=1.0, orientation='
    horizontal', description='blocked')
11 stopduration_slider= ipywidgets.IntSlider(min=1, max=1000, step=1,
    value=10, description='time for stop')
12 blocked_threshold= ipywidgets.FloatSlider(min=0, max=1.0, step=0.01,
    value=0.8, description='blocked threshold')
13
14 display(image_widget)
15
16 display(ipywidgets.HBox([blocked_slider, blocked_threshold,
    stopduration_slider]))
17
18 import math
19
20 angle = 0.0
21 angle_last = 0.0
22 count_stops = 0
23 go_on = 1
24 stop_time = 10 # The number of frames to remain stopped
25 x = 0.0
26 y = 0.0
27 speed_value = speed_control_slider.value
28
29 def execute(change):
30     global angle, angle_last, blocked_slider, robot, count_stops,
        stop_time, go_on, x, y, blocked_threshold
31     global speed_value, steer_gain, steer_dgain, steer_bias
```

```

32
33     steer_gain = steering_gain_slider.value
34     steer_dgain = steering_dgain_slider.value
35     steer_bias = steering_bias_slider.value
36
37     image_preproc = preprocess(change['new']).to(device)
38
39     #Collision Avoidance model:
40
41     prob_blocked = float(F.softmax(model_trt_collision(image_preproc),
42 dim=1).flatten()[0])
43
44     blocked_slider.value = prob_blocked
45     stop_time=stopduration_slider.value
46
47     if go_on == 1:
48         if prob_blocked > blocked_threshold.value: # threshold should
49 be above 0.5
50             count_stops += 1
51             go_on = 2
52         else:
53             #start of road following detection
54             go_on = 1
55             count_stops = 0
56             xy = model_trt(image_preproc).detach().float().cpu().numpy
57 ().flatten()
58             x = xy[0]
59             y = (0.5 - xy[1]) / 2.0
60             speed_value = speed_control_slider.value
61         else:
62             count_stops += 1
63             if count_stops < stop_time:
64                 x = 0.0 #set x steering to zero
65                 y = 0.0 #set y steering to zero
66                 speed_value = 0 # set speed to zero (can set to turn as
67 well)
68             else:
69                 go_on = 1
70                 count_stops = 0
71                 angle = math.atan2(x, y)
72                 pid = angle * steer_gain + (angle - angle_last) * steer_dgain
73                 steer_val = pid + steer_bias
74                 angle_last = angle
75                 robot.left_motor.value = max(min(speed_value + steer_val, 1.0),
76 0.0)
77                 robot.right_motor.value = max(min(speed_value - steer_val, 1.0),
78 0.0)
79
80 execute({'new': camera.value})
81
82 camera.observe(execute, names='value')
83
84 # 1.1 Python code robot.py
85 import time
86 import traitlets
87 from traitlets.config.configurable import SingletonConfigurable
88 from Adafruit_MotorHAT import Adafruit_MotorHAT
89 from .motor import Motor

```

```

84
85 class Robot(SingletonConfigurable):
86
87     left_motor = traitlets.Instance(Motor)
88     right_motor = traitlets.Instance(Motor)
89
90     # config
91     i2c_bus = traitlets.Integer(default_value=1).tag(config=True)
92     left_motor_channel = traitlets.Integer(default_value=1).tag(config=
True)
93     left_motor_alpha = traitlets.Float(default_value=1.0).tag(config=
True)
94     right_motor_channel = traitlets.Integer(default_value=2).tag(config
=True)
95     right_motor_alpha = traitlets.Float(default_value=1.0).tag(config=
True)
96
97     def __init__(self, *args, **kwargs):
98         super(Robot, self).__init__(*args, **kwargs)
99         self.motor_driver = Adafruit_MotorHAT(i2c_bus=self.i2c_bus)
100         self.left_motor = Motor(self.motor_driver, channel=self.
left_motor_channel, alpha=self.left_motor_alpha)
101         self.right_motor = Motor(self.motor_driver, channel=self.
right_motor_channel, alpha=self.right_motor_alpha)
102
103
104     def set_motors(self, left_speed, right_speed):
105         self.left_motor.value = left_speed
106         self.right_motor.value = right_speed
107
108     def forward(self, speed=1.0, duration=None):
109         self.left_motor.value = speed
110         self.right_motor.value = speed
111
112     def backward(self, speed=1.0):
113         self.left_motor.value = -speed
114         self.right_motor.value = -speed
115
116     def left(self, speed=1.0):
117         self.left_motor.value = -speed
118         self.right_motor.value = speed
119
120     def right(self, speed=1.0):
121         self.left_motor.value = speed
122         self.right_motor.value = -speed
123
124     def stop(self):
125         self.left_motor.value = 0
126         self.right_motor.value = 0
127
128 # 1.1.1 Python code motor.py
129 import atexit
130 from Adafruit_MotorHAT import Adafruit_MotorHAT
131 import traitlets
132
133 from traitlets.config.configurable import Configurable
134
135 class Motor(Configurable):

```

```

136
137     value = traitlets.Float()
138
139     # config
140     alpha = traitlets.Float(default_value=1.0).tag(config=True)
141     beta = traitlets.Float(default_value=0.0).tag(config=True)
142
143     def __init__(self, driver, channel, *args, **kwargs):
144         super(Motor, self).__init__(*args, **kwargs) # initializes
traitlets
145
146         self._driver = driver
147         self._motor = self._driver.getMotor(channel)
148         if(channel == 1):
149             self._ina = 1
150             self._inb = 0
151         else:
152             self._ina = 2
153             self._inb = 3
154         atexit.register(self._release)
155
156     @traitlets.observe('value')
157     def _observe_value(self, change):
158         self._write_value(change['new'])
159
160     def _write_value(self, value):
161
162         """Sets motor value between [-1, 1]"""
163         mapped_value = int(255.0 * (self.alpha * value + self.beta))
164         speed = min(max(abs(mapped_value), 0), 255)
165         self._motor.setSpeed(speed)
166         if mapped_value < 0:
167             self._motor.run(Adafruit_MotorHAT.FORWARD)
168             # The two lines below are required for the WaveShare Robot
Board only
169             self._driver._pwm.setPWM(self._ina,0,0)
170             self._driver._pwm.setPWM(self._inb,0,speed*16)
171         else:
172             self._motor.run(Adafruit_MotorHAT.BACKWARD)
173             # The two lines below are required for the WaveShare Robot
Board only
174             self._driver._pwm.setPWM(self._ina,0,speed*16)
175             self._driver._pwm.setPWM(self._inb,0,0)
176
177     def _release(self):
178         """Stops motor by releasing control"""
179         self._motor.run(Adafruit_MotorHAT.RELEASE)
180         # The two lines below are required for the WaveShare Robot
Board only
181         self._driver._pwm.setPWM(self._ina,0,0)
182         self._driver._pwm.setPWM(self._inb,0,0)
183
184 # 2. Python code detectnet.py
185 #!/usr/bin/env python3
186 import jetson.inference
187 import jetson.utils
188 import argparse
189

```

```

190 import sys
191
192 # parse the command line
193
194 parser = argparse.ArgumentParser(description="Locate objects in a live
195     camera stream using an object detection DNN.",
196     formatter_class=argparse.
197     RawTextHelpFormatter, epilog=jetson.inference.detectNet.Usage() +
198     jetson.utils.videoSource.Usage() +
199     jetson.utils.videoOutput.Usage() + jetson.utils.logUsage())
200
201 parser.add_argument("input_URI", type=str, default="", nargs='?', help=
202     "URI of the input stream")
203
204 parser.add_argument("output_URI", type=str, default="", nargs='?', help=
205     "URI of the output stream")
206
207 parser.add_argument("--network", type=str, default="ssd-mobilenet-v2",
208     help="pre-trained model to load (see below for options)")
209
210 parser.add_argument("--overlay", type=str, default="box,labels,conf",
211     help="detection overlay flags (e.g. --overlay=box,labels,conf)\
212     nvalid combinations are: 'box', 'labels', 'conf', 'none'")
213
214 parser.add_argument("--threshold", type=float, default=0.5, help="
215     minimum detection threshold to use")
216
217 is_headless = ["--headless"] if sys.argv[0].find('console.py') != -1
218     else [""]
219
220 try:
221     opt = parser.parse_known_args()[0]
222 except:
223     print("")
224     parser.print_help()
225     sys.exit(0)
226
227 # create video output object
228 output = jetson.utils.videoOutput(opt.output_URI, argv=sys.argv+
229     is_headless)
230
231 # load the object detection network
232
233 net = jetson.inference.detectNet(opt.network, sys.argv, opt.threshold)
234
235 # create video sources
236 input = jetson.utils.videoSource(opt.input_URI, argv=sys.argv)
237
238 # process frames until the user exits
239 while True:
240     # capture the next image
241     img = input.Capture()
242
243     # detect objects in the image (with overlay)
244     detections = net.Detect(img, overlay=opt.overlay)
245
246     # print the detections
247     print("detected {:d} objects in image".format(len(detections)))
248
249     for detection in detections:
250         print(detection)
251
252

```



```

237     # render the image
238     output.Render(img)
239
240     # update the title bar
241     output.SetStatus("{:s} | Network {:.0f} FPS".format(opt.network,
net.GetNetworkFPS()))
242
243     # print out performance info
244     net.PrintProfilerTimes()
245
246 # exit on input/output EOS
247     if not input.IsStreaming() or not output.IsStreaming():
248         break
249
250 import torch
251 device = torch.device('cuda')
252
253 import torch
254 from torch2trt import TRTModule
255
256 model_trt = TRTModule()
257 model_trt.load_state_dict(torch.load('best_steering_model_xy_trt.pth'))
    # well trained road following model
258
259 model_trt_collision = TRTModule()
260 model_trt_collision.load_state_dict(torch.load('best_model_trt.pth')) #
    well trained collision avoidance model
261
262 import torchvision.transforms as transforms
263 import torch.nn.functional as F
264 import cv2
265 import PIL.Image
266 import numpy as np
267
268 mean = torch.Tensor([0.485, 0.456, 0.406]).cuda().half()
269 std = torch.Tensor([0.229, 0.224, 0.225]).cuda().half()
270
271 def preprocess(image):
272     image = PIL.Image.fromarray(image)
273     image = transforms.functional.to_tensor(image).to(device).half()
274     image.sub_(mean[:, None, None]).div_(std[:, None, None])
275     return image[None, ...]
276
277 from IPython.display import display
278 import ipywidgets
279 import traitlets
280 from robot import Camera, bgr8_to_jpeg
281
282 camera = Camera.instance(width=224, height=224, fps=10)
283
284 image_widget = ipywidgets.Image()
285
286 traitlets.dlink((camera, 'value'), (image_widget, 'value'), transform=
    bgr8_to_jpeg)
287
288 from robot import Robot
289
290 robot = Robot()

```

9.2 RESULT

The working model of the robot was done and it was simulated to obtain the results successfully. Through this mini project we had recognized the general application of autonomous surveillance vehicle. We also studied the working principle of the model and its closed feedback loops. Jupyter Lab was used to control the robot through local network.



Figure 9.1: Results

9.3 PERFORMANCE EVALUATION

On experimentation, it was observed that the proposed robot model seems to be outperformed when compared to all different the surrounding objects. The following tasks are executed by the robot model

- Collision avoidance and road following
- Scanning surrounding for object detection

Chapter 10

CONCLUSION AND FUTURE SCOPE

It was observed on extermination that the proposed approach needs a vast training set for better accurate results; in the field of Surveillance and gather the surrounding data information. In all such cases, the proposed algorithm must be robust enough for accurate recognition of objects in the nearby places of the robot. The proposed approach can be further improvised through in cooperating weakly trained algorithms that can identify the objects with a minimum training data and also self-learning algorithms would aid in enhancing the accuracy of the algorithm and reduce the computational time.

We proposed a computerized method for the identification of nearby objects. In mobile surveillance system, our intention is to develop a remote controlled robot car capable of recognizing its surroundings. As a part of conclusion, this project offers the following facilities:

- DC Motor controlled car with superior controlling
- Jetson Nano board

Bibliography

- [1] Di Paola, D., & Milella, A. (2010, March). An autonomous mobile robotic system for surveillance of indoor environments. *IEEE Transactions on Robotics*, 26(2), 345-357. <https://doi.org/10.1109/TRO.2010.2042789>
- [2] Witwicki, S., & Castillo, J. C. (2017, September). Autonomous surveillance robots: A decision-making framework for networked multiagent systems. *IEEE Robotics & Automation Magazine*, 24(3), 89-98. <https://doi.org/10.1109/MRA.2017.2720483>
- [3] Valladares, S., & Toscano, M. (2021, January). Performance evaluation of the Nvidia Jetson Nano through a real-time machine learning application. *Journal of Real-Time Image Processing*, 18(1), 123-135. <https://doi.org/10.1007/s11554-020-01025-6>